Téma 9: Ukládání a serializace dat

Teoretický základ

Serializace je proces převodu složitých datových struktur v paměti (např. vnořených seznamů a slovníků) do bystového nebo textového formátu vhodného k uložení na disk, či k přenosu přes síť.

- **CSV (Comma Separated Values):** Modul `csv`. Ideální pro tabulková data z Excelu. Hodnoty oddělené znakem (čárkou nebo středníkem).
- **JSON:** Modul `json`. JavaScript Object Notation. Nejpoužívanější formát napříč jazyky na celém internetu. Do JSON se ukládají Python dictionaries (slovníky) přes funkci `json.dump()`.
- **Pickle:** Binární modul pouze pro Python. Umí uložit (zamrazit) i objekty vytvořené z vašich vlastních tříd.

Základní koncept k zamyšlení: Dump vs Dumps

Jaký je rozdíl mezi funkcemi `json.dump()` a `json.dumps()` (s písmenem "s" na konci)? Je to častý chyták! Ta s "s" Serializuje Do Stringu (převádí data na textový řetězec uchovaný čistě v paměti), zatímco první funkce provádí zápis přímo do otevřeného Souboru na disku.

Úkol 1 (Lehká obtížnost)

Zadání úkolu: V programu vytvořte Python slovník, který ukládá jméno a věk osoby. Tento slovník následně transformujte (tzv. serializujte) do formátu JSON a uložte jej pevně na disk do souboru s názvem `data.json` použitím knihovny `json`.

Řešení úkolu 1

```
import json

osoba_data = {
    "jmeno": "Jana",
    "vek": 24,
    "mesto": "Praha"
}

# Parametr 'w' a context manager pro bezpečný zápis na disk
with open("data.json", "w", encoding="utf-8") as f:
    # json.dump (zápis přímo do objektu souboru f)
    json.dump(osoba_data, f)
```

Úkol 2 (Střední obtížnost)

Zadání úkolu: Využijte vestavěný modul `csv` a vytvořte soubor `znamky.csv`. Pomocí speciálního zapisovače `csv.writer` vytvořte v tabulce dva oddělené řádky. Na prvním bude hlavička: *Předmět, Známka*. Na druhém řádku reálná data: *Fyzika, 2*.

Řešení úkolu 2

```
import csv

# newline='' řeší problém s prázdnými řádky navíc u Windows systémů
with open("znamky.csv", "w", newline='', encoding="utf-8") as f:

    zapisovac = csv.writer(f)

    # Zapisování po jednotlivých řádcích (vstupem je vždy seznam položek)
    zapisovac.writerow(["Předmět", "Známka"])
    zapisovac.writerow(["Fyzika", 2])
```

Úkol 3 (Těžká obtížnost)

Zadání úkolu: Máte velký a složitý konfigurační slovník. Chcete si jej nechat vypsat na konzoli, ale v pěkně odsazené (formátované) podobě, kterou dokáže přečíst i obyčejný uživatel, nikoliv jako slitou nudli. Použijte funkci `json.dumps()` s nastavením míry odsazení (indent).

Řešení úkolu 3

```
import json

# Velmi nepřehledná struktura na jednom řádku
konfigurace = {"server": "127.0.0.1", "port": 80, "logovani": {"chyby":
True}}

# json.dumps (se s na konci) vytváří textový String (neukládá na disk)
# parametr indent=4 zajistí krásné zarovnání a zalomení řádků
krasny_json = json.dumps(konfigurace, indent=4)

# Zobrazí se s perfektním formátováním
print(krasny_json)
```

Úkol 4 (Lehká obtížnost - dodatek)

Zadání úkolu: (Deserializace JSON) V prvním úkolu jsme ukládali data do `data.json`. Ukažte opačný postup: Otevřete tento soubor a načtěte jeho obsah (z formátu JSON) bezpečně rovnou do živé datové struktury "Python slovníku" využitím funkce `json.load()`.

Řešení úkolu 4

```
import json

# Pozor, pro pouhé čtení nastavujeme bezpečný mod 'r'
with open("data.json", "r", encoding="utf-8") as f:
    # Funkce load si data vysaje, přeloží struktury JSON syntaxe
    # a nám se vrátí připravený Dictionary.
    nacteny_slovnik = json.load(f)

# Ověříme tím, že se dotážeme na konkrétní klíč:
print(f"Z disku byla načtena osoba jménem: {nacteny_slovnik['jmeno']}")
```

Úkol 5 (Střední obtížnost - dodatek)

Zadání úkolu: K načítání složitých CSV tabulek existuje chytřejší třída `csv.DictReader`, která rovnou z prvního řádku pozná hlavičky a každý další řádek vrací jako napárovaný slovník. Založte si ukázkový CSV s hlavičkou (Jmeno,Vek) a vypište věky.

Řešení úkolu 5

```
import csv

# Představte si, že zde máme fyzický soubor klienti.csv s hodnotami.
with open("klienti.csv", "r", encoding="utf-8") as f:

    # DictReader na rozdíl od klasického readeru nevyhazuje prostá pole
["Pavel", "30"],
    # ale mapované slovníky {"Jmeno": "Pavel", "Vek": "30"}.
    ctenar = csv.DictReader(f)

    for radek in ctenar:
        print(f"Klient má {radek['Vek']} let.")
```

Úkol 6 (Těžká obtížnost - dodatek)

Zadání úkolu: (Pickle Serializace). JSON neumí uložit objekt vlastní třídy (pokud vytvoříte Třídu Kniha, do JSON ji přímo nenarvete). Modul `pickle` ale umí zamrazit zcela cokoliv, neboť to převádí na ryzí binární shluk. Vytvořte proměnnou `data = [1, 2, 3]` a uložte ji přes pickle v binárním Write módu (`wb`).

Řešení úkolu 6

```
import pickle

# Jakékoliv složité i objektové struktury
slozitosti = [1, 2, {"tajne": True}]

# Zásadní: mod "wb" znamená Write Binary (nezapisujeme znaky, ale jedničky
a nuly)
with open("zaloha.pkl", "wb") as f:
    # Stejná logika dump jako u JSONu (co, kam)
    pickle.dump(slozitosti, f)

print("Paměť byla zamrazena a uložena binárně na disk.")
```